

UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ingeniería Industrial y de Sistemas

Escuela Profesional de Ingeniería de Software

BANCO DE PREGUNTAS

Preparación para la Práctica Calificada 3

Tema: PL/SQL - Funciones y Procedimientos

Esquema de trabajo: VENTAS

Docente: Dr. Eric Gustavo Coronel Castillo

1. Propósito del banco de preguntas

Este banco contiene 10 preguntas orientadas a preparar al estudiante para la Práctica Calificada 3. Las preguntas trabajan el análisis técnico de código PL/SQL, la detección de errores lógicos, la identificación de errores en tiempo de ejecución y la construcción de funciones y procedimientos aplicados al esquema VENTAS.

El estudiante debe responder con precisión técnica, justificar cada decisión y construir código claro, mantenible y coherente con las reglas del negocio representadas por el esquema VENTAS.

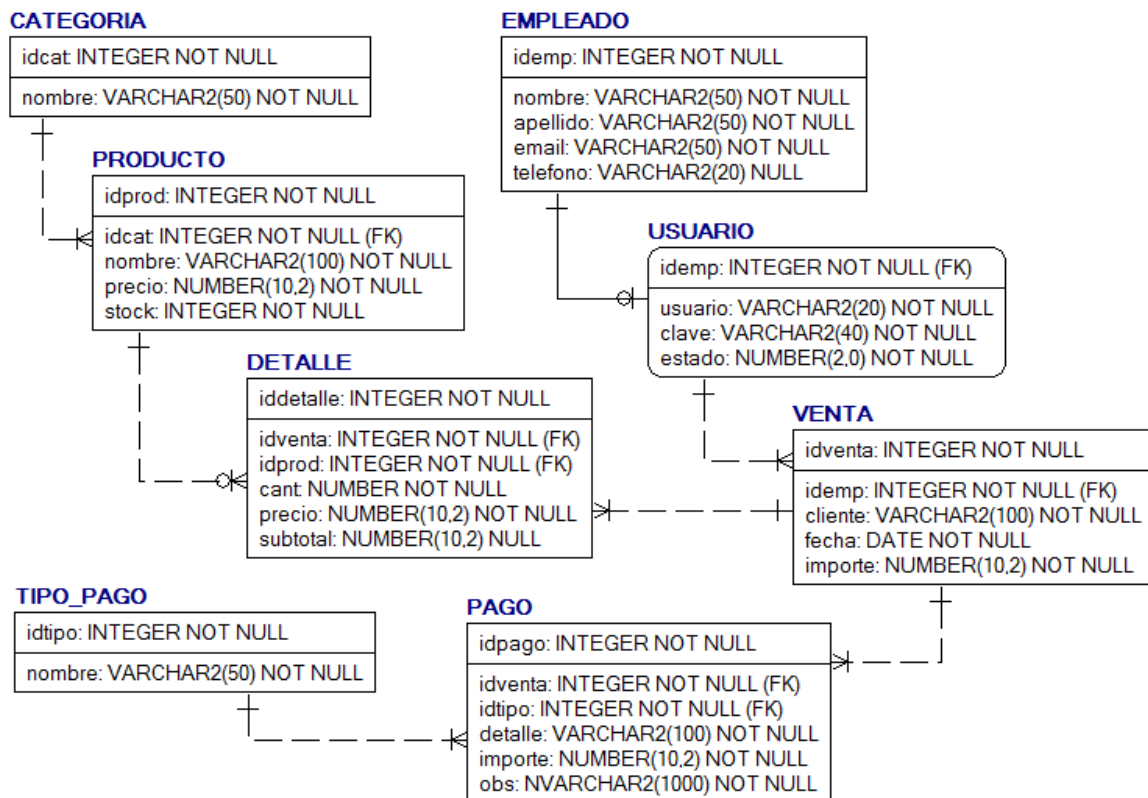
2. Indicaciones generales

- Trabaje sobre el esquema VENTAS y respete las tablas, claves primarias, claves foráneas y secuencias existentes.
- Use PL/SQL de Oracle: bloques, variables, SELECT INTO, sentencias DML, excepciones, funciones, procedimientos y parámetros.
- Cuando corresponda, declare variables usando %TYPE para mantener consistencia con las columnas del esquema.
- Use nombres claros para variables, funciones y procedimientos. Evite nombres ambiguos o iguales a columnas cuando puedan confundir la lectura del código.
- Cuando aplique reglas de negocio, utilice RAISE_APPLICATION_ERROR con mensajes claros.
- No incluya COMMIT ni ROLLBACK dentro de una función. En procedimientos, solo use control transaccional si el requerimiento lo solicita de forma explícita.
- La explicación técnica debe considerar el contexto del negocio y el impacto del error sobre la integridad de la información.

3. Esquema VENTAS - referencia mínima

Tabla	Uso principal
EMPLEADO	Registra los empleados que pueden estar asociados a una venta.
USUARIO	Registra usuarios asociados a empleados, con estado activo o inactivo.
CATEGORIA	Clasifica los productos del catálogo.
PRODUCTO	Registra productos, precio y stock disponible.
VENTA	Registra la cabecera de la venta: cliente, empleado, fecha e importe.
DETALLE	Registra los productos vendidos en cada venta, con precio, cantidad y subtotal.
TIPO_PAGO	Registra las formas de pago disponibles.
PAGO	Registra los pagos asociados a una venta.

Principales secuencias del esquema: SQ_EMPLEADO, SQ_CATEGORIA, SQ_PRODUCTO, SQ_VENTA, SQ_DETALLE y SQ_PAGO.



Modelo de referencia del esquema VENTAS.

Parte 1. Análisis de error lógico

En cada pregunta se presenta un bloque, función o procedimiento que compila, pero cuyo comportamiento no satisface correctamente la regla de negocio. El estudiante debe identificar los errores lógicos, explicar técnicamente su impacto y construir una versión corregida.

Pregunta 1. Cálculo del total de una venta

Contexto. En el esquema VENTAS, la tabla DETALLE almacena los productos vendidos en cada venta. Cada detalle contiene el precio aplicado, la cantidad vendida y el subtotal. Se requiere una función que devuelva el total de una venta a partir de sus detalles.

Código propuesto.

```
CREATE OR REPLACE FUNCTION fn_total_venta(  
    p_idventa IN venta.idventa%TYPE  
) RETURN NUMBER  
IS  
    v_total NUMBER(10,2);  
BEGIN  
    SELECT SUM(precio)  
    INTO v_total  
    FROM detalle  
    WHERE idventa = p_idventa;  
  
    RETURN NVL(v_total, 1);  
END;  
/
```

Se solicita al estudiante:

- Identifique los errores lógicos presentes en el código.
- Explique técnicamente por qué cada error afecta el resultado esperado dentro del contexto de una venta.
- Cree la versión correcta de la función, usando código claro y bien estructurado.

Pregunta 2. Registro de detalle y actualización de stock

Contexto. Cuando se agrega un producto a una venta, el sistema debe registrar el detalle de la venta y actualizar el stock del producto. El subtotal debe representar el importe correspondiente a la cantidad vendida.

Código propuesto.

```
CREATE OR REPLACE PROCEDURE pr_agrega_detalle_venta(  
    p_idventa IN detalle.idventa%TYPE,  
    p_idprod  IN detalle.idprod%TYPE,  
    p_cant    IN detalle.cant%TYPE  
)  
IS  
    vPrecio producto.precio%TYPE;  
    vStock  producto.stock%TYPE;  
BEGIN  
    SELECT precio, stock  
    INTO vPrecio, vStock  
    FROM producto  
    WHERE idprod = p_idprod;  
  
    IF vStock < p_cant THEN  
        RAISE_APPLICATION_ERROR(-20001, 'Stock insuficiente.');    END IF;  
  
    INSERT INTO detalle(iddetalle, idventa, idprod, precio, cant, subtotal)  
    VALUES(sq_detalle.NEXTVAL, p_idventa, p_idprod, vPrecio, p_cant, vPrecio);  
  
    UPDATE producto  
    SET stock = stock + p_cant  
    WHERE idprod = p_idprod;  
END;  
/
```

Se solicita al estudiante:

- Identifique los errores lógicos presentes en el procedimiento.
- Explique técnicamente el impacto de cada error sobre el importe de la venta y sobre el inventario.
- Cree la versión correcta del procedimiento, considerando validaciones mínimas de negocio.

Pregunta 3. Validación del saldo pendiente antes de registrar un pago

Contexto. Una venta puede tener uno o más pagos registrados. Antes de insertar un nuevo pago, el sistema debe verificar que el importe acumulado de los pagos de esa venta no supere el importe total de la venta.

Código propuesto.

```
CREATE OR REPLACE PROCEDURE pr_registra_pago_venta(
    p_idventa IN pago.idventa%TYPE,
    p_idtipo  IN pago.idtipo%TYPE,
    p_importe IN pago.importe%TYPE,
    p_detalle IN pago.detalle%TYPE,
    p_obs     IN pago.obs%TYPE
)
IS
    v_importe_venta venta.importe%TYPE;
    v_total_pagado  pago.importe%TYPE;
BEGIN
    SELECT importe
    INTO v_importe_venta
    FROM venta
    WHERE idventa = p_idventa;

    SELECT NVL(SUM(importe),0)
    INTO v_total_pagado
    FROM pago;

    IF (v_total_pagado + p_importe) < v_importe_venta THEN
        RAISE_APPLICATION_ERROR(-20002, 'El pago excede el saldo pendiente.');
```

```
    END IF;

    INSERT INTO pago(idpago, idventa, idtipo, detalle, importe, obs)
    VALUES(sq_pago.NEXTVAL, p_idventa, p_idtipo, p_detalle, p_importe, p_obs);
END;
```

```
/
```

Se solicita al estudiante:

- Identifique los errores lógicos presentes en el procedimiento.
- Explique técnicamente cómo estos errores afectan la validación del saldo pendiente.
- Cree la versión correcta del procedimiento, evitando que se registre un pago inválido.

Parte 2. Análisis de error en tiempo de ejecución

En cada pregunta se presenta código PL/SQL que puede compilar, pero que genera un error en tiempo de ejecución bajo determinadas condiciones. El estudiante debe explicar la causa técnica del error y construir una versión corregida.

Pregunta 4. Consulta del nombre de un producto

Contexto. Se requiere una función que retorne el nombre de un producto. En el catálogo existen productos cuyos nombres superan los 10 caracteres, por ejemplo, REFRIGERADORA o CAMISA COLOR VERDE.

Código propuesto.

```
CREATE OR REPLACE FUNCTION fn_nombre_producto(  
    p_idprod IN producto.idprod%TYPE  
) RETURN VARCHAR2  
IS  
    v_nombre VARCHAR2(10);  
BEGIN  
    SELECT nombre  
    INTO v_nombre  
    FROM producto  
    WHERE idprod = p_idprod;  
  
    RETURN v_nombre;  
END;  
/
```

Se solicita al estudiante:

- Indique qué error en tiempo de ejecución podría producirse al ejecutar la función con productos cuyo nombre excede la capacidad de la variable.
- Explique técnicamente la causa del error y su relación con la declaración de variables en PL/SQL.
- Cree la versión correcta de la función, usando buenas prácticas de declaración de tipos.

Pregunta 5. Obtención de productos asociados a una venta

Contexto. Una venta puede contener más de un producto. El procedimiento intenta mostrar el producto asociado a una venta, pero se ejecuta sobre una venta que tiene varios registros en DETALLE.

Código propuesto.

```
CREATE OR REPLACE PROCEDURE pr_producto_de_venta(  
    p_idventa IN venta.idventa%TYPE  
)  
IS  
    v_idprod detalle.idprod%TYPE;  
    v_nombre producto.nombre%TYPE;  
BEGIN  
    SELECT idprod  
    INTO v_idprod  
    FROM detalle  
    WHERE idventa = p_idventa;  
  
    SELECT nombre  
    INTO v_nombre  
    FROM producto  
    WHERE idprod = v_idprod;  
  
    DBMS_OUTPUT.PUT_LINE('Producto: ' || v_nombre);  
END;  
/
```

Se solicita al estudiante:

- Indique qué error en tiempo de ejecución se produciría cuando la consulta devuelve más de una fila.
- Explique por qué SELECT INTO exige un resultado unitario en este contexto.

- Cree la versión correcta del procedimiento, usando un cursor explícito o un bucle FOR implícito para recorrer todos los productos de la venta.

Pregunta 6. Inserción de un producto repetido en una misma venta

Contexto. La tabla DETALLE tiene una restricción de unicidad sobre la combinación IDVENTA e IDPROD. El procedimiento se ejecuta intentando registrar dos veces el mismo producto dentro de la misma venta.

Código propuesto.

```
CREATE OR REPLACE PROCEDURE pr_inserta_detalle_directo(  
    p_idventa IN detalle.idventa%TYPE,  
    p_idprod  IN detalle.idprod%TYPE,  
    p_precio  IN detalle.precio%TYPE,  
    p_cant    IN detalle.cant%TYPE  
)  
IS  
BEGIN  
    INSERT INTO detalle(iddetalle, idventa, idprod, precio, cant, subtotal)  
    VALUES(sq_detalle.NEXTVAL, p_idventa, p_idprod, p_precio, p_cant, p_precio * p_cant);  
END;  
/
```

Se solicita al estudiante:

- Indique qué error en tiempo de ejecución se produciría al violar la restricción de unicidad.
- Explique técnicamente por qué la base de datos rechaza la operación.
- Cree una versión correcta del procedimiento, validando previamente la existencia del producto en el detalle o actualizando la cantidad si ya existe.

Parte 3. Creación de funciones y procedimientos

En esta parte se presentan requerimientos de negocio. El estudiante debe construir la función o el procedimiento solicitado, aplicando buenas prácticas de PL/SQL, validaciones de negocio y manejo de excepciones cuando corresponda.

Pregunta 7. Crear la función `fn_stock_producto`

Contexto. El área de ventas necesita consultar de forma rápida el stock disponible de un producto antes de ofrecerlo al cliente. Se requiere una función llamada `fn_stock_producto` que reciba el identificador del producto y retorne el stock actual registrado en la tabla `PRODUCTO`. Si el producto no existe, la función debe generar un error de aplicación con un mensaje claro.

Se solicita al estudiante:

- Cree la función `fn_stock_producto`.
- Use `PRODUCTO.IDPROD%TYPE` y `PRODUCTO.STOCK%TYPE` para declarar parámetros y variables.
- Controle el caso de producto inexistente mediante una excepción adecuada.
- El código debe ser claro, ordenado y mantenible.

Pregunta 8. Crear el procedimiento `pr_crear_venta`

Contexto. Cuando se inicia una venta, primero se registra la cabecera en la tabla `VENTA`. En ese momento todavía no existen detalles, por lo que el importe inicial debe ser cero. Se requiere un procedimiento llamado `pr_crear_venta` que reciba el nombre del cliente y el identificador del empleado, registre la venta con la fecha actual y devuelva el `IDVENTA` generado.

Se solicita al estudiante:

- Cree el procedimiento `pr_crear_venta` con un parámetro `OUT` para devolver el `IDVENTA` generado.
- Valide que el cliente no sea nulo ni vacío.
- Valide que el empleado exista en la tabla `EMPLEADO`.
- Use la secuencia `SQ_VENTA` para generar el identificador de la venta.
- Inserte la venta con `FECHA` igual a `SYSDATE` e `IMPORTE` igual a 0.
- No confirme la transacción dentro del procedimiento, salvo que sustente técnicamente lo contrario.

Pregunta 9. Crear el procedimiento `pr_agregar_producto_venta`

Contexto. Una venta puede tener varios productos. Se requiere un procedimiento que agregue un producto a una venta existente. El procedimiento debe validar la existencia de la venta, la existencia del producto, que la cantidad sea positiva y que exista stock suficiente. Luego debe insertar el detalle, descontar el stock y actualizar el importe total de la venta.

Se solicita al estudiante:

- Cree el procedimiento `pr_agregar_producto_venta`.
- Use parámetros para `IDVENTA`, `IDPROD` y cantidad.
- Obtenga el precio y stock del producto desde la tabla `PRODUCTO`.
- Valide stock suficiente antes de insertar el detalle.
- Inserte en `DETALLE` usando `SQ_DETALLE`.
- Calcule correctamente el `SUBTOTAL` como `PRECIO * CANTIDAD`.
- Actualice `PRODUCTO.STOCK` descontando la cantidad vendida.
- Actualice `VENTA.IMPORTE` sumando el subtotal agregado.
- Maneje errores de negocio con mensajes claros.

Pregunta 10. Crear la función `fn_saldo_venta`

Contexto. El área de caja necesita conocer el saldo pendiente de una venta. El saldo se obtiene restando al importe total de la venta la suma de los pagos registrados en la tabla `PAGO`. Si la venta no tiene pagos, el total pagado debe considerarse cero.

Se solicita al estudiante:

- Cree la función `fn_saldo_venta`.
- La función debe recibir el `IDVENTA` y retornar el saldo pendiente.
- Use `NVL` o `COALESCE` para tratar correctamente ventas sin pagos.
- Valide que la venta exista antes de calcular el saldo.
- No modifique datos dentro de la función.
- Maneje el caso de venta inexistente con un error de aplicación.

Criterios sugeridos para la autoevaluación del estudiante

- La respuesta identifica el problema técnico y no se limita a decir que el código está mal.
- La explicación relaciona el error con una regla de negocio del esquema VENTAS.
- La versión corregida compila y respeta la estructura de las tablas.
- El código usa nombres claros, tipos basados en columnas y manejo de excepciones cuando corresponde.
- La solución evita efectos secundarios innecesarios y conserva la integridad de la venta, el detalle, el stock y los pagos.